When a screened pattern search spends more simulator calls: evidence from mineral-processing circuit cases

Felipe Santibáñez-Leal, ORCID 0000-0002-0150-3246

Preprint · v1.2, 2 November 2026 · Licence CC BY 4.0
CAOS open-research programme · mineral-processing optimization
Version DOI 10.5281/zenodo.23895650 · concept DOI (always latest) 10.5281/zenodo.23833134
Not peer reviewed. Source code, data and computational records: github.com/fsantibanezleal/CAOS_OreFlow

Abstract

Surrogate screening can add trial evaluations as well as avoid them. We evaluate a neural-surrogate search step, with a reconstruction guard and a Gaussian-process interval-width filter, inside multistart generalized pattern search with a progressive barrier. Its unscreened control uses the same objective, starts, bounds, stop rules and circuit evaluator. Across 72 target-mode variants from twelve authored mineral-processing cases, screening recorded 26,367 distinct cached trial states versus 22,487 without screening, a 17.25% increase (case-cluster bootstrap 95% interval 9.28 to 25.88%). Counts were higher in 68 variants, tied in four and lower in none. Terminal outcomes matched in 65 pairs; the other seven were different feasible decisions, with at most 0.0027 t/h difference in recovered metal. Of 5,811 exact-evaluated proposals, 901 improved an incumbent. Their mean absolute recovery error was 0.71 percentage points (case-cluster interval 0.49 to 0.98), with a 38.16-point maximum. A separate twelve-case nominal ablation recorded 3,683 evaluations without screening, 3,753 with one-step screening and 3,974 with one-and-two-step screening. These finite simulator results do not establish global optimality, elapsed-time savings or performance on operating plants.

1 Introduction

Grinding, classification and flotation decisions interact through nonlinear balances, equipment limits and product-quality constraints. Direct simulation can be expensive when an optimizer repeatedly evaluates a complete flowsheet. A learned approximation may reduce that cost by ranking candidates before exact simulation, but only if the number of avoided simulator calls exceeds the number of additional calls caused by the screening rule. Prediction accuracy alone is insufficient: optimization depends on errors near competing candidates and on the geometry of the feasible region.

Surrogate models have been used to approximate phenomenological comminution circuits. Koh et al. report a neural-network surrogate for simulator throughput with substantial speedup on their studied circuit and data design [13]. More generally, surrogate management methods combine approximate models with exact evaluations and retain convergence safeguards for the original problem [5]. Generalized pattern search provides derivative-free search with convergence results under specified smoothness and direction-set assumptions [25, 3]. These results motivate guarded screening, but do not imply that a particular screen reduces calls or preserves a particular finite-budget solution.

This study evaluates one integrated procedure on OreFlow's committed case records. It asks two bounded questions: does screening reduce exact engine evaluations, and how closely do the screened and unscreened searches agree on feasibility and recovered metal? The cases are authored scenarios rather than measured operating data. The result is therefore an algorithmic comparison within a declared simulator domain, not a plant-performance claim.

Three distinctions govern the experiment. First, approximation error and optimization cost are separate outcomes. A screen can predict well and still spend more calls if its chosen points fail to improve the current search state. Second, agreement of two finite-budget runs is not a certificate of optimality: both can terminate at the same local point, and either can stop with no feasible point under the fixed evaluation budget. Third, a model's uncertainty estimate is useful for screening only if its calibration carries to the exact states that the optimizer proposes. The study retains each question as a separate measurement instead of treating a favorable endpoint as evidence that the approximation was safe or economical.

The experiment is paired at the variant level. Each screened run and its unscreened comparator share the case, operating point, objective weight, decision bounds, deterministic starts, feasibility rule and termination settings. The two runs use independent evaluation caches; within each run, a repeated state is counted once. We therefore measure additional distinct states exposed to the exact engine, not a ratio of runtimes that might change with parallel scheduling, model loading or cache warmness. The pairing controls the declared inputs and search configuration, while the finite cases limit external validity.

The contributions are a reproducible account of the implemented screen; a 72-pair estimate of cached exact-evaluation change across twelve authored cases; a decision-agreement analysis that distinguishes seven different feasible decision vectors from two matched infeasible outcomes; and a recorded error analysis of all 5,811 exact-evaluated proposals. A separate twelve-case nominal experiment isolates the search-step radius. Four selected circuits receive an additional filter-removal experiment. Both studies are descriptive evaluations of this code and its saved model export. Neither establishes a new convergence theorem, a plant recommendation, or a reduction in elapsed computation time.

2 Related work

Koh et al. trained a deep neural model to reproduce a phenomenological comminution-circuit simulator and used the approximation to accelerate scenario evaluation [13]. That work addresses approximation of simulator outputs. Here, the object of evaluation is the effect of a surrogate-driven search step inside a bounded derivative-free optimizer, including the exact-call cost and terminal decision relative to an unscreened control.

The surrogate management framework formalizes the interplay between approximation models and exact objective evaluations [5]. Generalized pattern search methods offer convergence properties for suitable objective functions and generating directions; those properties do not automatically transfer to a modified search step without the relevant assumptions and proof [25, 3]. Accordingly, this experiment reports finite-suite behavior and makes no global-convergence claim for the screened implementation.

Trust-region model-management methods restrict model use to a local region and compare actual with predicted decrease; the radius update is part of the validity argument, rather than a standalone uncertainty threshold [5, 15, 9, 18]. The present method has a different contract: it proposes at one and two mesh steps from current feasible and infeasible incumbents, then sends only the highest-ranked surviving candidate to the exact circuit. It does not build a local interpolation set, compute a trust-region ratio, or require a model-improvement bound. Its heuristic filters are empirical guards, not substitutes for model-adequacy conditions.

Generalized pattern search and mesh adaptive direct search derive stationarity results from their polling directions, mesh refinement, constraint treatment and regularity assumptions [25, 2, 3, 1]. Progressive barriers provide a way to compare feasible and infeasible incumbents without replacing constraints by an arbitrary fixed penalty [17, 8]. In OreFlow, the exact poll remains present, but a successful screen proposal can become the next incumbent and alter the subsequent poll sequence. That finite perturbation requires a separate implementation-specific convergence analysis; this paper does not infer the classical guarantees for the modified algorithm.

Bayesian optimization uses a probabilistic surrogate and an acquisition rule to balance predicted reward and uncertainty, often with expensive black-box objectives [11, 23, 22, 7]. Safe Bayesian optimization adds explicit assumptions and mechanisms for keeping queried states inside an admissible set; SafeOpt's guarantees, for example, depend on a Gaussian-process regularity model and a known safe seed [24]. OreFlow's GP interval is used only as a reject rule. It is not an acquisition function and does not bound the neural model's pointwise error. A narrow GP interval also does not establish that a circuit state is physically safe: all admissibility

checks and final process results still come from the exact engine.

Predictive uncertainty requires its own evaluation. A GP's posterior variance is conditional on the chosen kernel, likelihood and training design; empirical coverage can depart from nominal coverage under misspecification or distribution shift [19, 16, 12, 14]. Conformal methods offer marginal coverage statements under exchangeability, but those statements also depend on the sampling protocol and do not imply conditional coverage in optimizer-selected regions [4]. The present benchmark records interval coverage on its learning splits and pointwise residuals at selected proposals. It does not claim a formal coverage guarantee for adaptive proposals.

Evaluation unit also matters. Randomly held-out states from a common simulator design primarily measure interpolation, while grouped or spatial holdouts test a different transfer target. Structured data can make random splits optimistic when correlated observations from one group cross the train/test boundary [20, 6]. OreFlow's twelve named cases and ore-group splits are authored simulator constructs, not independent mines; those group labels let the evaluator ask whether scores change when a circuit case is withheld, not whether the method generalizes to a population of operating plants.

These distinctions locate this paper's narrow contribution. Earlier work supplies the optimizer class, surrogate models, uncertainty methods and validation cautions; this paper evaluates one concrete coupling among them under matched exact-engine budgets. Its negative cost result does not contradict acceleration reported for other objectives, models or simulators. Instead, it supplies a counterexample to the assumption that inserting a cheap predictor into direct search necessarily saves exact evaluations. The per-variant records and one-step ablation identify the observed mechanism and make the result testable against later screen designs. The computer-experiment literature emphasizes that surrogate quality depends on the design points and the intended prediction task, not just the regression family [21]. Proper scoring rules likewise require a defined predictive target and sampling process [10]. These perspectives support reporting proposal error and evaluation cost separately: neither a smooth approximation nor a narrow interval is a sufficient statistic for optimizer performance.

3 Method

3.1 Process decision problem

The search varies the grind target within the case contract and, for circuits with flotation, the collector dose and rougher gas velocity. The evaluator returns recovered primary metal M in t/h, recovery, concentrate grade, total specific energy E in kWh/t, power and water use. For a selected recovery weight $w \in [0, 1]$, the normalized objective is

$$J(x; w) = w\frac{M(x)}{M_0} - (1 - w)\frac{E(x)}{E_0}, \tag{1}$$

where M_0 and E_0 are the values at the variant's base operating point. Grade, installed power and water limits are treated as relaxable constraints through a progressive barrier; contract bounds on decisions are unrelaxable. The paired searches use the same fixed objective weight, starts and stopping rules. The subsequent warm-started weight-path calculation is a separate descriptive study.

The decision vector is expressed in physical units at the service boundary and transformed to normalized coordinates for mesh movement. Each case activates only controls supported by its process configuration. Flotation circuits use target product size, collector dosage and superficial gas velocity; a non-flotation circuit may use target size and classifier cut. The response $M(x)$ is payable-metal production, not a grade proxy, and $E(x)$ is total specific energy. Base values normalize objectives across cases with different feed rates and ore grades. This normalization changes the tradeoff scale but does not make the result an economic objective: prices, operating costs, capital and production schedules do not enter the function.

Let the engineering constraints be $g_j(x) \leq 0$, including minimum concentrate grade and upper bounds on installed power and water intensity. Feasible points have zero aggregate violation. Infeasible progress is represented by a progressive-barrier state, with objective and violation compared jointly. The process contract's lower and upper control limits define the search domain, so the optimizer does not infer permission to extrapolate beyond the authored operating envelope. This matters to screening because candidate features outside the

learning domain should be refused by the reconstruction guard, while the exact evaluator retains its own contract and physical-state checks.

For objective weight $w = 1$, the primary study maximizes recovered metal subject to declared constraints. For $w < 1$, energy contributes to a normalized tradeoff, but the 72-pair cost aggregate concerns the locked weight saved in each target-mode variant. The later weight path is excluded from that denominator because it is a different sequence of warm-started problems. These accounting boundaries prevent a search run from being mixed with additional sensitivity calculations that have different costs and stopping behavior.

3.2 Exact and screened searches

The baseline is a generalized pattern search with a progressive feasibility barrier. The screened procedure uses the same engine, objective, starts, mesh, bounds, constraints and stopping budget. At each iteration, the exported neural evaluator predicts candidate responses one and two mesh steps from feasible and infeasible incumbents. A proposal is eligible only when its reconstruction error is below the learned guard threshold and the Gaussian-process 95% recovery half-width does not exceed five percentage points. That interval belongs to the Gaussian process, not to the neural prediction used for ranking, and neither filter guarantees accuracy. The eligible candidate with the highest predicted objective is evaluated by the exact circuit engine. Only an engine result can update an incumbent or become a reported endpoint; the ordinary poll remains available when the search step does not improve it.

Each case record contains screen counts, accepted proposals, the surrogate and engine results for each proposal, the evaluation count, the terminal outcome and a paired unscreened search from the same starts. This paired design isolates the computational effect of the screen within the simulator and its finite search settings. It does not estimate runtime on a plant control system.

The filter is applied in a fixed order to make each recorded rejection reproducible. Contract-invalid points are excluded by the optimizer before learned inference. For remaining candidates, the reconstruction guard is checked first, then the GP half-width, then duplicate-cache status and predicted objective ranking. This order determines which rejection counter receives a candidate that would fail multiple predicates. Candidate records preserve the score and interval needed to audit the decision. Neither threshold is tuned during the search: adaptive threshold selection would define a different policy and invalidate the paired comparison. The exact numeric thresholds are serialized with the model export and form part of the experiment identity.

The reconstruction head and response head have separate responsibilities. The former is a state-support proxy and does not predict recovered metal; the latter predicts process outputs used for ranking. The serialized network is loaded once per optimizer invocation, and inference is batched over the finite stencil. Predictions are transformed back to process units before computing the ranking objective. The exact circuit output remains authoritative for constraint violation, incumbent updates and reported endpoints. Thus the surrogate can order or reject candidate states, but cannot supply a terminal process result.

In ordinary direct search, poll points are evaluated according to the active mesh and a successful point updates the incumbent. Here, the selected screen proposal is evaluated exactly before the ordinary poll is allowed to proceed. If that proposal succeeds, the changed incumbent becomes the next stencil center; if it fails, the poll can still find a successful point. This insertion adds a branch to the state machine. A successful proposal may replace later polling work, or may create a longer trajectory. Cost cannot be estimated by multiplying proposal count by one unit: cache hits, incumbent movement, mesh updates and barrier comparisons all alter the unique-state total. The paired cache counters therefore measure completed search work more directly than prediction counts, though they still do not represent wall-clock time.

Proposal records are nested under optimizer starts, preserving information about interactions among multistart trajectories. The case remains the analysis cluster because variants within it share process assumptions. The generator refuses to proceed unless all 72 pairs are present, computes decision agreement from exact saved coordinates and compares proposal predictions with exact engine values. It does not recreate an endpoint from surrogate outputs. These provenance properties make the method auditable without implying that auditability itself establishes efficiency or accuracy.

The model receives a fixed feature vector from the learning pipeline. Each coordinate is standardized using

training-set mean μ_j and scale s_j:

$$z_j = (x_j - \mu_j)/s_j, \qquad j = 1, \ldots, p. \tag{2}$$

The feature order, scaling constants and target transformations are serialized with the ONNX network. A candidate outside the process contract is refused before model inference; a candidate inside the contract may still be outside the training distribution and is assessed by the guard and interval rule. The learned screen does not enlarge the optimizer's feasible domain.

The reconstruction network maps standardized features through a latent representation and returns $\widetilde{z}$. Its score and pass indicator are

$$r_g(z) = p^{-1} \sum_{j=1}^{p} (z_j - \widetilde{z}_j)^2, \qquad I_g(z) = \mathbb{1}[r_g(z) \le \tau_g]. \tag{3}$$

The threshold τ_g is fixed in the model export. Low reconstruction error means the autoencoder reproduces a point under its trained representation; it does not prove small response error, closeness in each scientific feature, or membership in a convex support set. An autoencoder can map a shifted input to a familiar reconstruction. Feature-wise shifts and nearest-row distances therefore remain visible in the persisted candidate record, although they do not alter the deployed Boolean rule in this experiment.

The GP filter is fit to recovery targets during model training. For training design Z, covariance matrix K_θ, target vector y, and numerical noise λ, the persisted factors satisfy

$$\alpha = (K_\theta + \lambda I)^{-1} y, \qquad L L^\top = K_\theta + \lambda I. \tag{4}$$

For candidate z, let k_z be its vector of kernel covariances with training rows. The posterior quantities use the standard conditional Gaussian equations:

$$\mu_{GP}(z) = k_z^\top \alpha, \tag{5}$$

$$\sigma^2_{GP}(z) = k_\theta(z, z) + \lambda - k_z^\top (K_\theta + \lambda I)^{-1} k_z. \tag{6}$$

The saved Cholesky factor allows a triangular solve without refactorizing the training covariance at each candidate. The screen's two-sided half-width is $h(z) = q\sigma_{GP}(z)$, with release multiplier q, and its filter is

$$I_u(z) = \mathbb{1}[h(z) \le \tau_u], \qquad \tau_u = 5 \text{ recovery percentage points.} \tag{7}$$

The GP mean does not rank candidates; the neural surrogate supplies ranking targets. The GP is a separate veto channel, not a calibrated prediction interval around the neural network.

Candidate construction is deterministic. Let x_F and x_I denote feasible and infeasible centers, e_j the coordinate basis, Δ the mesh step and $S = \{1, 2\}$. The bounded candidate stencil is

$$C_k = \text{unique}\{x_c + s\Delta e_j : \ c \in \{F, I\}, \ s \in S, \ j = 1, \ldots, d, \ \pm, \ x_c + s\Delta e_j \in \mathcal{B}\}. \tag{8}$$

Feasible centers precede infeasible centers; one-step precedes two-step; coordinates follow contract order; positive precedes negative. Previously evaluated exact states are removed by the search cache. This order resolves ties reproducibly and is part of the method specification.

For each candidate, the procedure computes the neural targets, guard score and GP half-width. The admissible proposal set is

$$A_k = \{x \in C_k : I_g(z(x)) I_u(z(x)) = 1\}. \tag{9}$$

The surrogate objective $\widehat{J}$ transforms predicted recovery into recovered-metal units through the base state's fixed feed ratio, which is invariant for the selected control variables. The first maximizer under the deterministic stencil order is submitted to the exact engine:

$$x_s = \text{first} \arg\max_{x \in A_k} \widehat{J}(x). \tag{10}$$

The screen does not authoritatively predict constraint satisfaction. The exact response determines barrier acceptance, incumbent updates and reported values. If no candidate passes, the normal poll runs. If one passes, exactly one screen-selected point is evaluated; if that exact point fails to improve, the normal poll still follows.

This ordering creates the testable cost mechanism. A proposal is charged before its acceptance is known. A non-improving proposal consumes an exact simulation and does not replace a poll point. An improving proposal can replace some polling work, but changes the incumbent and therefore the subsequent path. A cheap prediction is consequently insufficient to establish a cheaper search. The paired comparison counts unique exact trial keys for each run, with independent caches:

$$\Delta N_i = N_{i,\text{screen}} - N_{i,\text{plain}}, \qquad \Delta N_{\text{suite}} = \sum_{i=1}^{72} \Delta N_i. \tag{11}$$

Negative values mean fewer unique exact states with screening; positive values mean added work; zero is a tie. The counters exclude uncached endpoint audits, the later weight path, model fitting, model loading, inference latency and parallel contention.

For dimension d, iteration count I and at most $2d|S|$ candidate coordinates per center, stencil and predicate work scales as $O(Id|S|)$ before duplicate and bound removal. Neural inference is linear in the number of stored layer weights. The GP uses kernel distances and a triangular solve with the fixed factor; its training factorization is paid once during export. One exact evaluation includes closed-circuit comminution and unit-balance checks, so its cost can dominate one model pass. This per-call contrast motivates the screen but cannot predict net calls; the experiment measures the count difference directly.

Four controls govern the deployed screen: stencil distance set S, guard threshold τ_g, interval threshold τ_u, and the underlying mesh schedule and budget. The release uses $S = \{1, 2\}$ and $\tau_u = 5$ recovery points. The guard threshold and normalization are saved in the model artifact and are not tuned on the 72 outcomes. A separate nominal study changes only S to $\{1\}$. The filter-removal experiment forces one or both predicates to pass while leaving the prediction, exact acceptance, objective and stopping settings intact. These are diagnostic interventions rather than policies selected after observing the main test set.

The code is divided between `data-pipeline/pipeline/methods/screen.py`, which reconstructs the exported network and GP calculations, and `data-pipeline/pipeline/methods/optimization.py`, which owns ordering, cache, barrier and exact acceptance. The browser parity implementation is `frontend/src/learning/screen.ts`; its agreement tests are an implementation check, not an independent scientific replication. The experiment in this paper uses the committed Python bake records.

Algorithm 1 Guarded surrogate search step with exact incumbent acceptance

Require: Incumbents x_F, x_I, mesh Δ, screen s, exact evaluator f, limits τ_g, τ_u

$C \leftarrow$ ordered in-bounds candidates at configured mesh distances
$A \leftarrow \emptyset$
for all $x \in C$ not already evaluated **do**
    $z \leftarrow \text{features}(x)$; $r \leftarrow \text{reconstructionError}(z)$
    if $r \le \tau_g$ **then**
        $(\mu, \sigma) \leftarrow \text{surrogateAndGP}(z)$
        if $q\sigma \le \tau_u$ **then**
            append $(x, \widehat{J}(x))$ to A
        end if
    end if
end for
if $A \neq \emptyset$ **then**
    $x_s \leftarrow$ first maximizer of predicted objective in A
    $y_s \leftarrow f(x_s)$
    if y_s improves an incumbent under the exact barrier rule **then**
        return updated exact incumbent and success
    end if
end if
evaluate unchanged ordered poll and return its exact barrier update

Algorithm 1 locates the learned model's role: it orders proposals, while the engine decides acceptance and output. It also exposes why a false-positive screen can increase cost. The chosen exact trial is charged before

the algorithm knows whether it will improve, and a failed proposal leaves the ordinary poll intact. The persisted per-start records retain the information needed to count those events, although they do not include a timed trace of every inference operation.

An exact proposal can affect two cost quantities differently: the total number of distinct states and the number of unique states additionally introduced by the screen. With paired baseline count N_i^0 and screened count N_i^s, define the case-weighted suite contrast as

$$\Delta_{\text{suite}} = 100 \left(\frac{\sum_i N_i^s}{\sum_i N_i^0} - 1 \right), \tag{12}$$

which weights each variant by its baseline workload and is not the arithmetic mean of variant percentages. This is the statistic used in the abstract and results. It is reported beside the direction count and cluster interval so the reader can distinguish aggregate burden from consistency across variants. The counters are cache-specific; they do not include model fitting, inference latency, uncached audits or elapsed time.

The implementation's exact-engine boundary is also the boundary for feasibility. A surrogate prediction cannot turn an infeasible state into an admissible incumbent, even if its predicted objective is attractive. When the exact evaluation reports constraint violation, the progressive-barrier comparison updates its infeasible reference according to the optimizer's violation ordering. When a feasible point is found, it competes under the objective among feasible incumbents. The screen therefore changes which candidate is evaluated and when, while retaining the original exact constraint contract. This preserves an important safety property of the software interface but does not amount to physical plant safety certification.

4 Theory and scope

The analysis here is a cost identity and a conditional mechanism statement, not a convergence theorem. Consider one iteration at which the screened and plain searches have the same incumbents, mesh, cache and poll stencil. If the screen proposes an exact state that is not already cached and that state does not improve either incumbent, the screened procedure incurs one additional unique exact evaluation before it executes the same poll. Under these conditions its next cache count exceeds the plain count by one at that iteration. This follows directly from the execution order: exact evaluation precedes the unchanged poll, and the failed proposal cannot remove a poll evaluation. If the proposal improves, the next incumbent and candidate sequence can change, so no stepwise count identity extends to the remainder of the run.

This conditional statement explains a plausible source of overhead without predicting the suite-wide sign. It separates (i) the chance that a model-selected point is useful, (ii) whether its exact result replaces poll work, and (iii) the size of the later path change. A high fraction of accurate predictions does not determine any of these quantities. Conversely, a proposal with moderate pointwise error can still improve the barrier or objective if its ranking is favorable. The accepted state is exact in either case; the question is whether spending that exact evaluation changed subsequent search work in a beneficial direction.

Let P_i be the set of screen proposals actually sent to the engine in paired run i, and U_i the unique exact poll states. In the special case where every proposal fails and the plain and screened poll trajectories remain identical, the count relation is $N_{i,screen} = N_{i,plain} + |P_i$
$setminus U_i|$. The expression counts only proposals not already in the screened cache. Once any proposal succeeds, this equality no longer follows because the two trajectories may diverge. The release records allow the total paired difference to be measured, but they do not identify a counterfactual trajectory after each successful proposal. We therefore report the observed ΔN_i rather than attributing all added or saved calls to one local cause.

Classical generalized pattern-search results depend on the poll direction set, mesh refinement, constraint conditions and continuity or nonsmoothness assumptions for the objective. The OreFlow search step inserts a surrogate-selected exact candidate ahead of the original poll; if accepted, it selects a different center. Although the engine still evaluates every candidate that can become incumbent and the original poll remains after a non-improving proposal, these facts alone do not establish that all classical assumptions or proof steps still apply. The implementation has finite stop budgets and fixed direction ordering, and its progressive-barrier

update is itself an explicit simplification. No global or local stationarity guarantee is claimed for the combined release implementation.

The uncertainty filter also supplies no guarantee without a calibration premise. A nominal GP interval has its usual interpretation only under a suitable model and data-generating assumptions. The proposal states are adaptively chosen using another model, so they need not be exchangeable with random held-out learning rows. Neither the GP posterior width nor autoencoder reconstruction error yields a deterministic bound on neural recovery error in this experiment. The empirical proposal errors are therefore a direct diagnostic, while the five-point threshold is only the frozen heuristic used by the evaluated implementation.

5 Experimental design

The primary test set consists of 72 target-mode variants drawn from twelve authored mineral-processing cases. Six target-mode variants per case vary the objective weight and operating settings under the case contract. A further 24 classifier-cut variants are outside the screen's feature coverage and remain unscreened; they are not included in the paired estimate. The twelve cases span rougher flotation, desliming, gravity and magnetic-separation circuit families. Their outputs are simulated from authored parameters, not observed production. Case identity and ore-group labels are retained so results can be inspected by evaluation unit.

The two procedures share the same deterministic initial operating point and declared interior starts for each variant. They also share control bounds, exact process evaluator, objective normalization, process constraints, mesh initial and minimum sizes, opportunistic poll order, progressive-barrier update, maximum engine-evaluation budget and feasible endpoint audit. The sole treatment is the availability of the learned search step. The unscreened comparator follows the same generalized pattern search with that step disabled. Its exact-state cache is independent of the screened cache, while repeats within a run are deduplicated. All model and objective settings are frozen before evaluating the paired artifacts.

We state four empirical hypotheses. H1 predicts a lower total distinct exact-evaluation count under screening. H2 predicts no material change in feasible endpoint decisions, assessed by exact vector identity, terminal status and recovered-metal difference. H3 predicts that accepted proposal errors are concentrated within the filter's recovery-width scale. H4 predicts that limiting the candidate stencil to one mesh step reduces the additional evaluation cost relative to the one-and-two-step screen. H1 is the primary efficiency hypothesis; H2 is a descriptive agreement hypothesis, not a noninferiority test because no tolerance or power calculation was specified before the bake. H3 is not a coverage claim, because the filter width belongs to a separate GP model rather than the ranked neural network.

The primary cost statistic is the paired difference in unique exact evaluations. We report suite totals, variant directions (more, tied, fewer), and family aggregates. The headline percentage is (
$sum_i N_{i,screen}/$
$sum_i N_{i,plain} - 1)$
$times 100$. Because six variants from a case share authored ore and circuit assumptions, variants are not treated as independent replicates. Uncertainty intervals will resample the twelve cases as clusters, retaining all six variants whenever a case is drawn. This interval describes variation across the authored case set; it is not a population confidence interval for real mines.

Decision outcomes are measured separately. Terminal statuses distinguish an optimal feasible point from the optimizer's finite-budget infeasible status. Feasible decision vectors are identical when every declared coordinate agrees to a numerical tolerance of 10^{-12}; recovered-metal differences are computed from the exact endpoint values. Proposal error is the absolute difference between predicted and exact recovery, in percentage points, for each selected proposal that reaches the engine. Candidate rejections are counted by guard and interval reason. We report medians, upper quantiles and maxima alongside counts because the long tail can be consequential even when central error is small.

The additional radius experiment uses twelve nominal case states at fixed recovery weight $w = 1$ and compares plain, one-step and full one-and-two-step search. It was executed with the committed model export under a separate measurement script and output JSON. The filter-removal experiment is run on four cases selected to represent soft copper flotation, hard copper flotation, fine magnetite and free-milling gold. It compares no

screen, full screen, guard removed, interval filter removed, and both removed. The small, purposefully varied set diagnoses component behavior but is not used to estimate suite-wide efficiency. All variants use the same model export and exact engine; the policy changes only the stated predicate.

The case records, screen export, model weights and experiment scripts are versioned in the repository. The bake's random design and training seeds are part of the committed learning artifact. The paired optimizer comparison itself is deterministic under a fixed source version, model bytes and floating-point implementation. The compute resource is a consumer NVIDIA RTX 4070 laptop GPU for the released-model learning bake, with CPU workers for case simulation and studies; the optimizer's exact search and these finite filter ablations run on CPU. The paper does not use GPU utilization as a cost metric. The release records preserve software version, contract digest and exact model references, while elapsed times are excluded from the main comparison because the bake's process-level timing does not isolate optimizer-only wall time.

5.1 Evaluation set and measures

The baked suite contains 96 variants: 72 target-mode variants and 24 classifier-cut variants. The latter are intentionally unscreened because the learned screen's features do not cover that circuit mode. Both searches find no feasible endpoint in two target-mode iron-magnetite variants under their declared settings. We report paired evaluation counts over all 72 target-mode records and compare feasible decisions for the 70 records where both searches find one. A matching infeasible status counts as a matching terminal outcome; it is neither a feasible endpoint nor a proof of global infeasibility.

The primary cost measure compares the recorded `evaluations` counters after the fixed-weight multistart searches. Each is the number of distinct trial states stored in that search's cache; repeated states across starts count once. Screened and unscreened runs use separate caches. Refused trial states remain recorded outcomes. These counters exclude uncached base/endpoint audits, the subsequent weight path, fitting, loading and screening inference, so they do not instrument every simulator invocation or measure wall-clock cost. Secondary measures are per-variant count ordering, terminal-outcome agreement, recovered-metal difference, number of proposals that improve the incumbent and absolute recovery error at proposed points. Percentage-point errors are differences between recovery percentages, not relative percentage errors.

6 Results

6.1 Exact evaluation cost

Across the 72 paired target-mode variants, screening recorded 26,367 cached trial evaluations; the baseline recorded 22,487. The relative increase is

$$100\left(\frac{26{,}367}{22{,}487}-1\right)=17.25\%. \tag{13}$$

Screening cost more in 68 variants and tied in four; it saved recorded evaluations in none. The comparison includes the fixed-weight multistart run for each variant. The counters are captured before the separate weight-path study. They are not a wall-clock benchmark and exclude the one-time training and export cost of the surrogate. Figure 1 plots each pair against equality: values above the diagonal are additional screen cost, which occurs for 68 variants. The horizontal aggregate in Table 1 is the ratio of summed counts, not the mean of per-variant percentages. That statistic weights cases by their unscreened search workload and answers the suite-level resource question directly. Table 2 shows that the positive total is not confined to a single circuit family; its family totals are descriptive because cases within each family share authored modeling choices.

The case-level distribution in Figure 2 shows how the same suite result varies across authored circuits. Positive medians and spread above zero indicate added calls in a case, while negative observations would identify a case where proposals displaced more polling work than they added. The figure is not a sample of independent ore bodies: its twelve horizontal units are the authored cases, and six operating variants within each case remain correlated. We therefore supplement the sum ratio with a case-cluster bootstrap interval in Table 1, resampling complete cases rather than pretending that all 72 variants are exchangeable.

Figure 6 places exact endpoint agreement beside cost change. A small recovered-metal difference does not imply a small cost difference: the two outcomes measure separate properties of the coupled search. Table 1 reports 65 identical terminal outcomes, consisting of 63 identical feasible vectors and two matching finite-budget infeasible statuses. The remaining seven outcomes are distinct feasible vectors; their recovered-metal differences remain below the stated maximum. Neither the cluster interval nor endpoint agreement establishes equivalence on a wider case population.

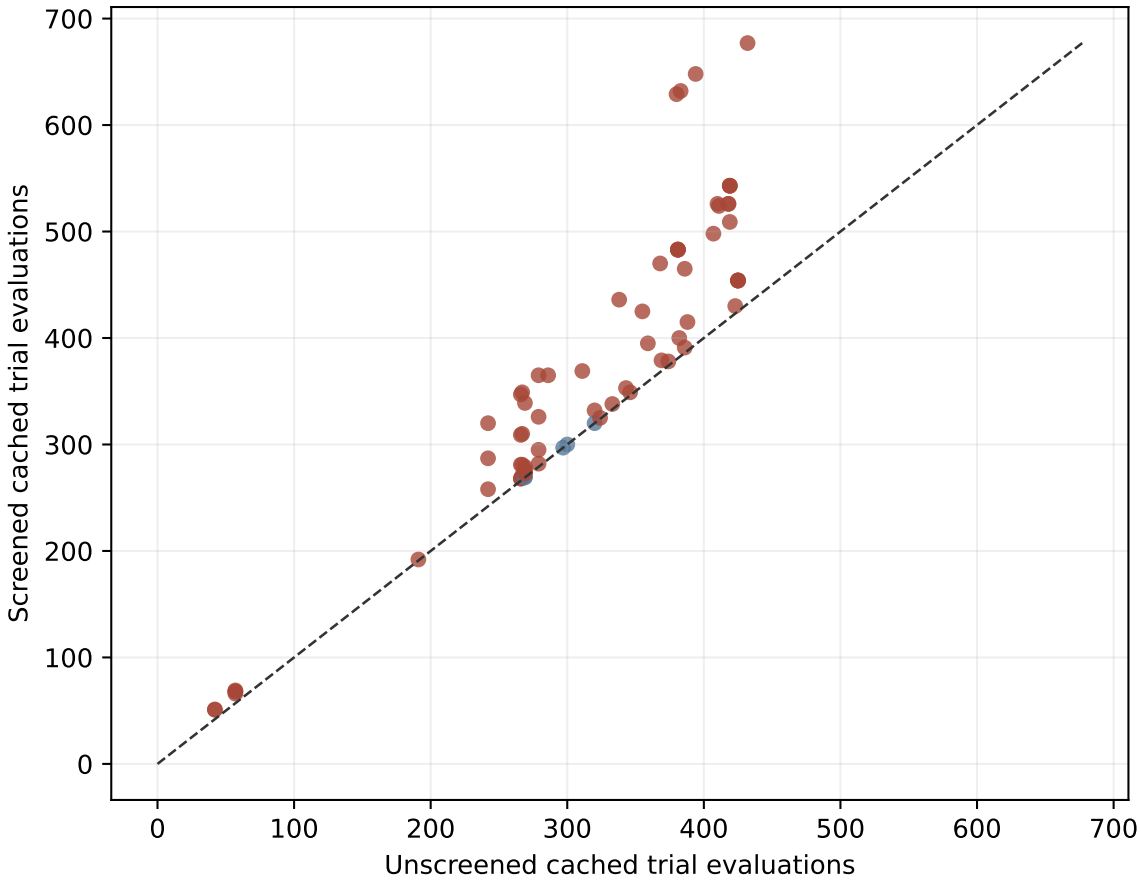


Figure 1: Paired cached exact-engine evaluations for each target-mode variant; points above equality indicate added screened work.

6.2 Decision agreement

Both procedures find feasible endpoints in 70 variants and return infeasible search statuses in the remaining two. They return the same terminal outcome in 65 of 72 variants: the 63 exactly matching feasible decision vectors plus the two matching infeasible statuses. Seven feasible variants select different decision vectors. Across those seven, the largest absolute difference in recovered primary metal is 0.002653 t/h, or 0.0168% relative to the larger value. Five favor the unscreened result in recovered metal and two favor the screened result. This close numerical agreement on these cases does not show that screening is safe for other inputs, constraints or stopping budgets.

Decision-vector equality and objective-value agreement are different outcomes. Two process settings may deliver similar recovered metal while differing in grind target, reagent dose or gas velocity, with different energy and grade implications. The equality statistic is strict and the response difference is separate, so close but operationally distinct settings are not called identical. The infeasible pairs also remain distinct from feasible agreement: two searches that stop without a feasible endpoint share a status, but that does not certify that the

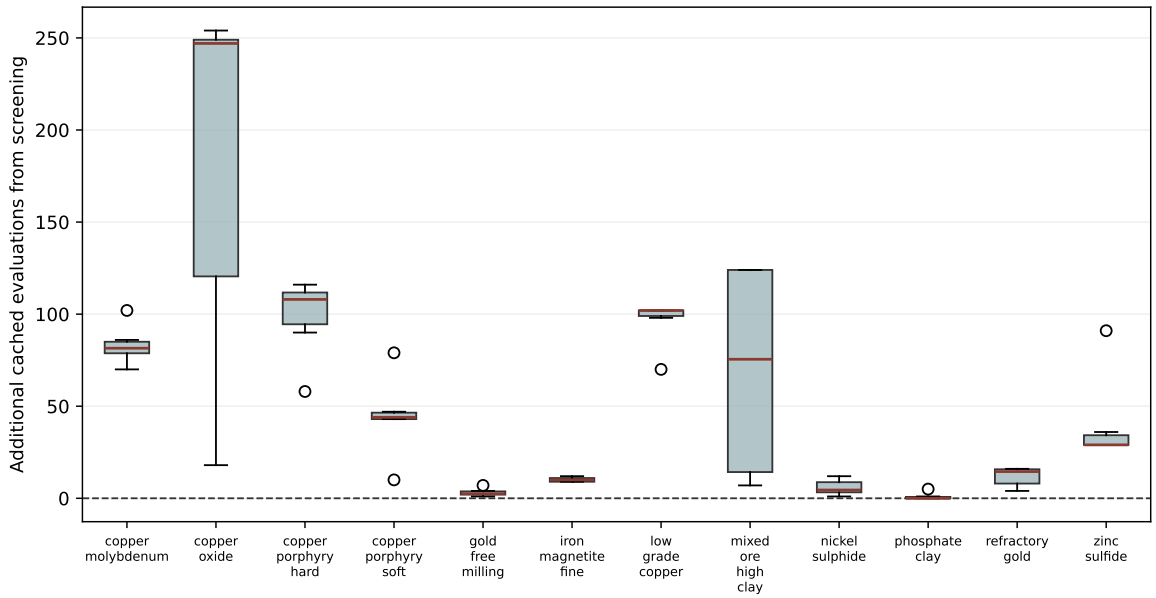


Figure 2: Variant-level evaluation-count changes by authored case and circuit family; positive values mean screening increased exact-engine work.

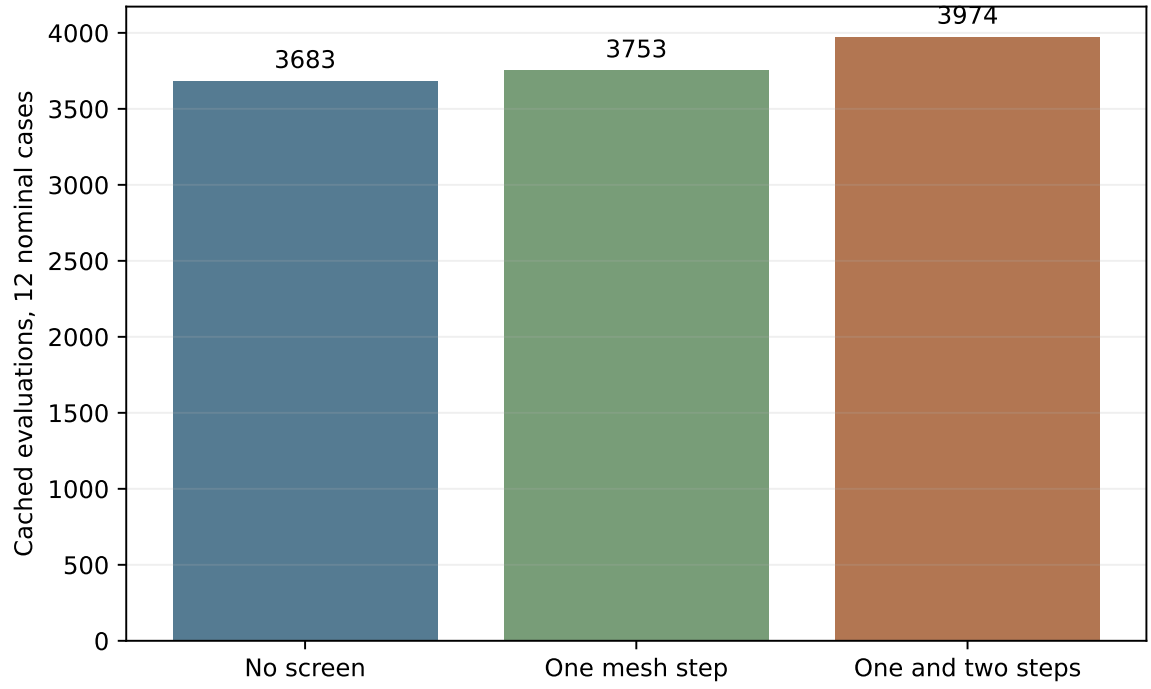


Figure 3: Nominal weight-one evaluation totals across twelve authored cases under no screen, one-step screening and the full one-and-two-step policy.

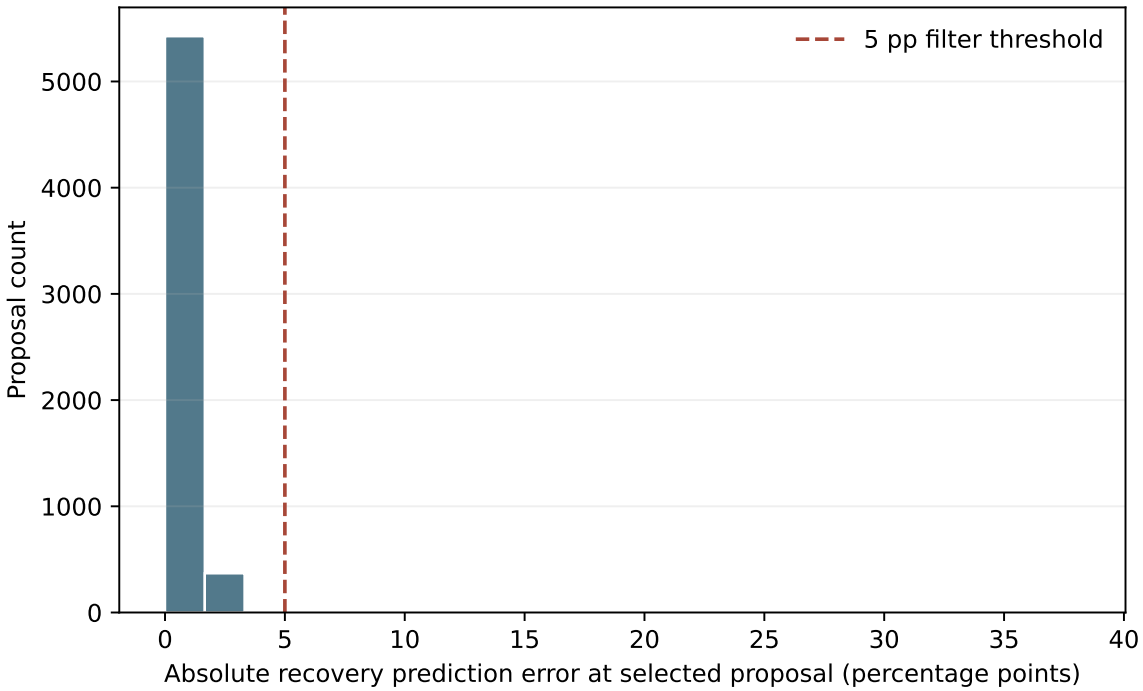


Figure 4: Absolute recovery prediction error for every exact-evaluated selected proposal, with the five-point interval filter threshold marked.

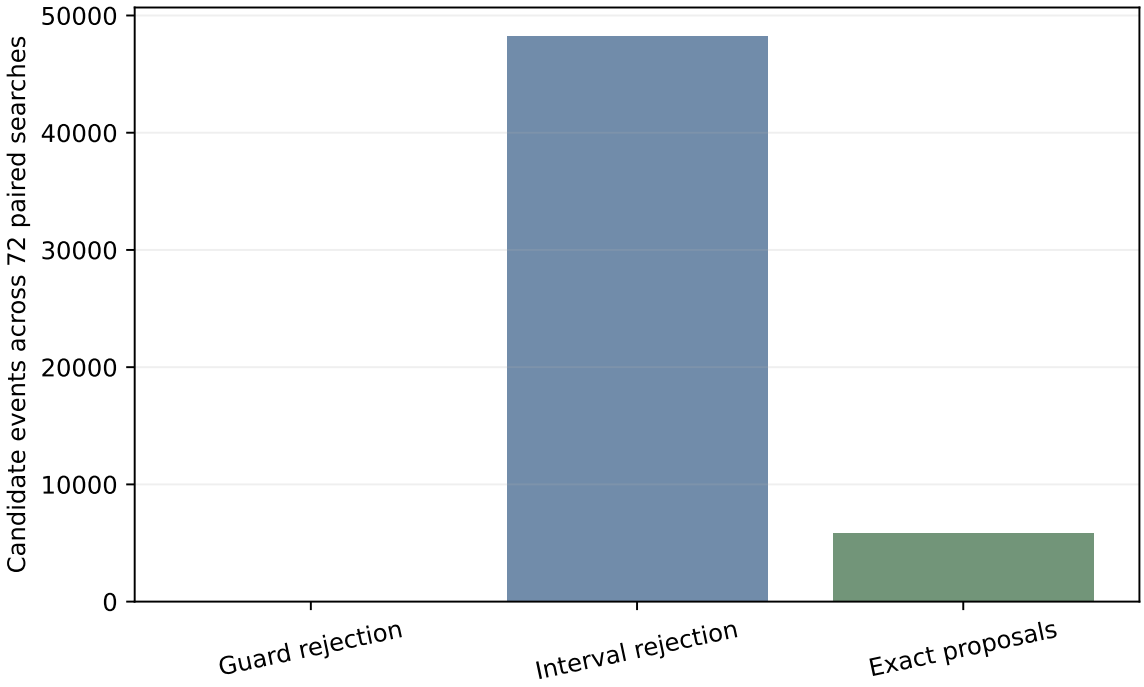


Figure 5: Guard and interval rejections and proposals aggregated over the 72 paired variants; rejection totals count candidate events.

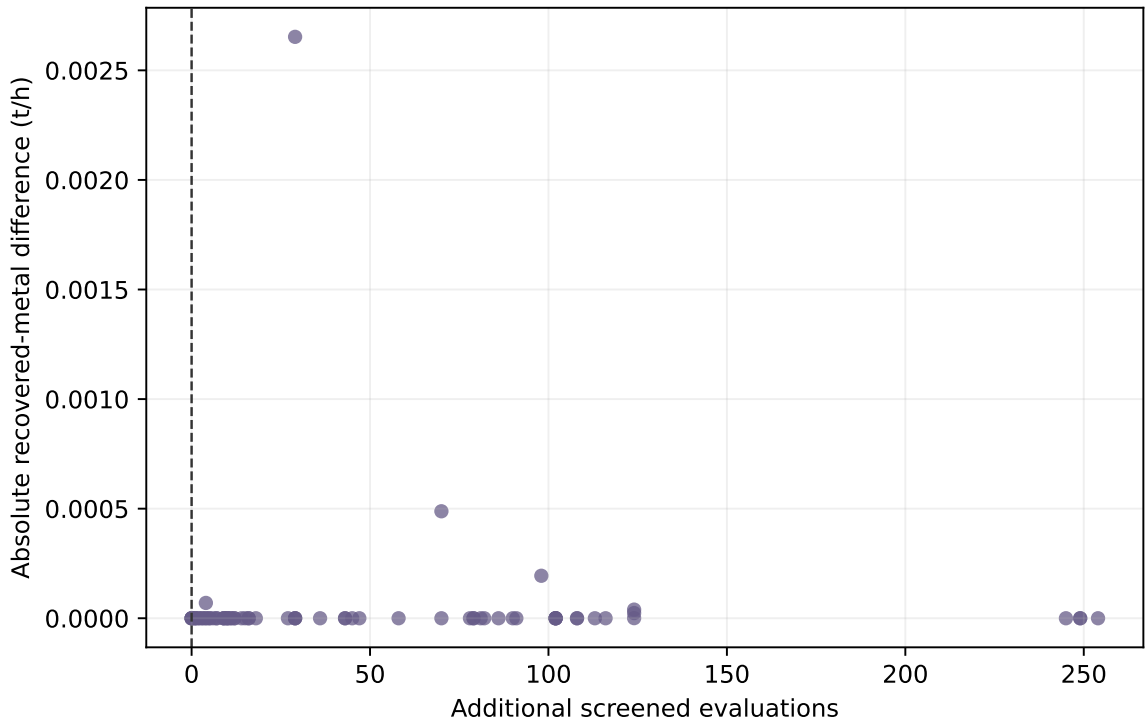


Figure 6: Added exact evaluations plotted against absolute recovered-metal differences for the matched target-mode outcomes across all twelve authored cases.

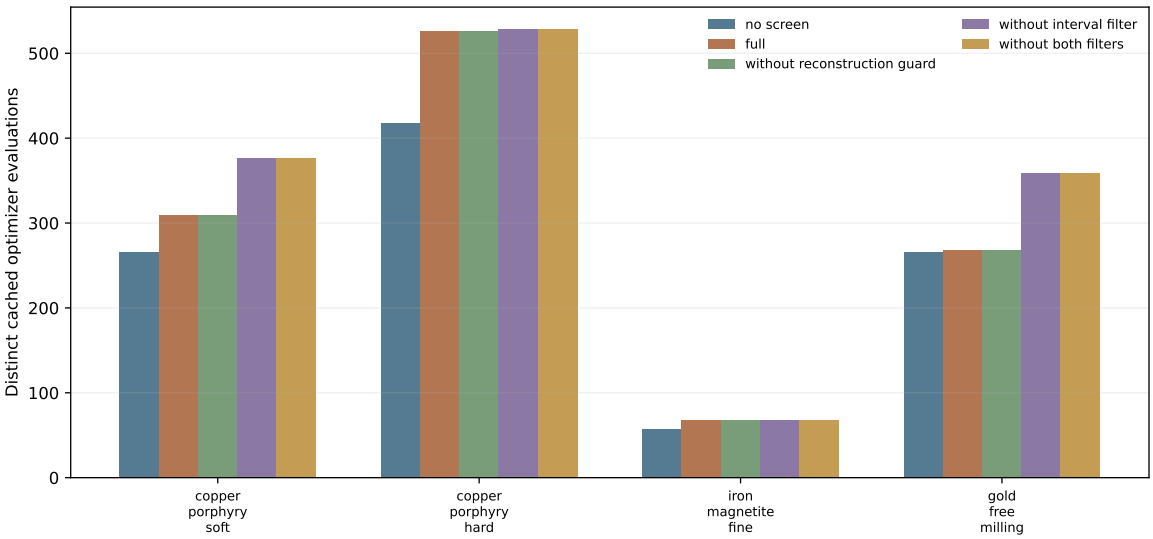


Figure 7: Exact evaluation counts for each of five filter policies across four authored cases selected to span distinct circuit configurations.

Table 1: Paired search outcomes for the 72 authored target-mode variants, using distinct cached exact-engine trial states as the cost measure.

Measure	Result
Paired variants	72
Screened evaluations	26367
Unscreened evaluations	22487
Change	+17.25%
Case-cluster bootstrap 95% interval for change	[9.28, 25.88]%
Higher / tied / lower cost	68 / 4 / 0
Same terminal outcome	65 / 72
Case-cluster bootstrap 95% interval for agreement	[80.6, 98.6]%
Median recovered-metal difference (t/h)	0.000000
Maximum recovered-metal difference (t/h)	0.002653

Table 2: Evaluation counts aggregated by the four authored circuit families; every family retains its number of paired target-mode variants.

Circuit family	Variants	Unscreened	Screened	Change (%)
deslime_rougher	18	6240	6697	7.32
gravity_rougher	6	1616	1635	1.18
magnetic	6	312	373	19.55
rougher	42	14319	17662	23.35

Table 3: Nominal weight-one search ablation on twelve authored cases, isolating candidate distance from the matched unscreened baseline.

Screen policy	Cases	Evaluations	Change vs baseline	Same recovered metal
No screen	12	3683	+0.00%	12
One mesh step	12	3753	+1.90%	12
One and two steps	12	3974	+7.90%	11

Table 4: Screen candidate counts and exact proposal diagnostics aggregated from the 72 paired artifacts, including the selected-proposal error distribution.

Event	Count
Guard rejections	19
Interval-width rejections	48268
Exact proposals	5811
Proposals improving incumbent	901
Absolute error median (pp)	0.483
Mean absolute error (pp), case-cluster 95% interval	0.712 [0.487, 0.975]
Absolute error 95th percentile (pp)	1.825
Maximum absolute error (pp)	38.158

Table 5: Filter-removal ablation totals across four selected authored cases, compared with the matched no-screen policy and exact endpoint agreement.

Policy	Evaluations	Change vs no screen	Same endpoint
no screen	1007	+0.00%	4/4
full	1171	+16.29%	4/4
without reconstruction guard	1171	+16.29%	4/4
without interval filter	1332	+32.27%	4/4
without both filters	1332	+32.27%	4/4

Table 6: Per-case filter policy outcomes preserve evaluation counts, recovered metal, and endpoint agreement for all twenty matched runs.

Authored case	Policy	Evaluations	Recovered metal (t/h)	Endpoint
copper porphyry soft	no screen	266	5.195311	same
copper porphyry soft	full	309	5.195311	same
copper porphyry soft	without reconstruction guard	309	5.195311	same
copper porphyry soft	without interval filter	377	5.195311	same
copper porphyry soft	without both filters	377	5.195311	same
copper porphyry hard	no screen	418	2.548871	same
copper porphyry hard	full	526	2.548871	same
copper porphyry hard	without reconstruction guard	526	2.548871	same
copper porphyry hard	without interval filter	528	2.548871	same
copper porphyry hard	without both filters	528	2.548871	same
iron magnetite fine	no screen	57	232.893406	same
iron magnetite fine	full	68	232.893406	same
iron magnetite fine	without reconstruction guard	68	232.893406	same
iron magnetite fine	without interval filter	68	232.893406	same
iron magnetite fine	without both filters	68	232.893406	same
gold free milling	no screen	266	0.000878	same
gold free milling	full	268	0.000878	same
gold free milling	without reconstruction guard	268	0.000878	same
gold free milling	without interval filter	359	0.000878	same
gold free milling	without both filters	359	0.000878	same

underlying process problem has no feasible solution.

The 65-of-72 match is descriptive, not a reliability level. The variants share twelve case configurations, and no independent operational validation set exists. A future decision study should define a multivariate acceptance region before evaluating results, including tolerances for active controls, recovery, grade, energy and constraint margins. It should report the fraction of new cases inside that region with uncertainty clustered at the ore or plant level. The present saved outcomes can help design such a protocol but cannot replace it with a post hoc metal difference.

6.3 Proposal behavior and prediction errors

The screened search records 79,034 candidate evaluations by the screening logic. Nineteen are rejected by the learned-state guard and 48,268 by the uncertainty-width rule. The algorithm sends 5,811 selected proposals to the engine; 901, or 15.5%, improve the incumbent. For those evaluated proposals, the mean absolute recovery error is 0.712 percentage points, the median is 0.483 points and the maximum is 38.158 points. The large maximum is material: the width filter did not eliminate every large pointwise error in the proposal record. The evidence does not support describing the surrogate as uniformly accurate where it is used.

Table 4 and Figure 5 distinguish candidate rejection from exact evaluation. Rejection counts are candidate events, so one optimizer iteration can contribute multiple rejections; they are not independent observations and should not be added to the cache count. The 5,811 exact proposals represent the points selected by the policy, whereas 79,034 is the screen's candidate-processing count. This distinction matters operationally: a cheap inference stage can inspect many points while still sending one to the circuit, but the proposal that it selects adds work unless it replaces an ordinary poll that would otherwise have been evaluated. The realized engine counter, rather than the number of neural predictions, is the efficiency outcome used here.

The error distribution is strongly right-tailed. Its median is below one percentage point, but the maximum is more than seven times the five-point GP half-width threshold. That apparent discrepancy does not mean the GP arithmetic is internally inconsistent: the GP interval filters candidates through its own recovery model, while the neural network supplies the selected prediction, and the reported error compares the latter with the exact recovery. The two fitted predictors do not share a calibration guarantee. Figure 4 places every selected

proposal in the distribution, and Table 4 reports the tail as well as the average. A deployment rule based only on the mean would hide the largest error and would be an unsound interpretation of these records.

The proposal-level file records show that only 901 proposals improve their incumbent, about 15.5 percent of selected exact proposals. That rate is an outcome of this deterministic search and its policy, not a probability estimate for future decisions. The statistic does not establish that the other proposals were useless: a non-improving exact evaluation can still affect poll status, mesh progression or later search. Conversely, a proposal that improves the current incumbent can alter subsequent work and is not necessarily responsible for a better final solution. These records support a mechanism discussion, but not a causal decomposition of every added evaluation.

7 Ablation

The component experiments ask whether either screen predicate or candidate radius explains the measured behavior. The evidence uses two distinct designs. The twelve-case nominal study isolates radius at recovery weight one, while the four-case filter study removes the reconstruction guard, the interval-width veto, or both from otherwise matched executions. They are not pooled: the first covers a larger set with one operating point per case, and the second examines policy mechanics on four deliberately selected circuit configurations. This design can diagnose whether a rule is necessary for the observed behavior in those records; it cannot estimate an average effect for every one of the 72 paired variants.

The radius results are generated from `docs/design/features/live-optimizer/measurements/screen_variants.json`. No-screen search totals 3,683 distinct evaluations across twelve nominal cases. Restricting screening to one mesh step gives 3,753, an increase of 1.90 percent; the implemented one-and-two-step stencil gives 3,974, an increase of 7.90 percent. The ordered totals answer H4 in the expected direction: removing the farther proposals reduces the count relative to the full screen. It does not reverse the primary finding, because even the one-step policy remains above the unscreened count. Table 3 reports the totals and endpoint metal agreement, and Figure 3 makes the increase visible without implying that counts are wall-clock time.

This radius comparison is a policy ablation rather than a one-factor causal estimate of candidate quality. Reducing the stencil changes which proposal can become the next incumbent, which changes subsequent centers, mesh states and cache hits. The trajectories therefore diverge after a different decision. The observed 221-evaluation difference between full and one-step screening combines the extra candidates and downstream path changes; it cannot be interpreted as the direct cost of exactly two-step predictions alone. It does show that the farther stencil did not earn back its additional search work on this nominal set.

The filter-removal outcomes are summarized in Table 5, visualized by case in Figure 7, and itemized in Table 6.

The filter-removal study completed all five policies on each of the four selected cases. The no-screen runs used 1,007 evaluations in total, the full policy used 1,171 (+16.29

The component counts reveal different roles for the two predicates in this subset. The reconstruction guard rejected no candidate in any of the four full-policy runs, so removing it had no measured effect; this ablation provides no evidence about its behavior on cases that actually trigger it. The interval filter rejected 1,791 candidate events under the full policy. Removing it increased exact proposals from 299 to 541 and added 161 cached evaluations, while the four endpoints remained unchanged. The veto therefore reduced proposal and exact-call load relative to the no-interval variant here, but it did not make screened search cheaper than the matched no-screen baseline. Its contribution is a reduction in the screen's own added workload, not a demonstrated overall saving relative to polling.

The unchanged endpoints do not make the interval filter redundant. Evaluation count is the measured resource outcome, and extra exact proposals can be costly even when the selected optimum remains the same. Conversely, the lower count under the filter does not prove calibration: the GP interval is not a probabilistic guarantee for the neural ranking error, and the proposal-error analysis in the larger suite includes a 38.158-point maximum. These four runs identify a specific local tradeoff. They do not establish the interval veto as a safe or universally efficient rule, nor do they show that its removal would preserve decisions on untested cases.

The study also separates inference load from simulator load. The interval predicate can reject many proposals

before exact evaluation, while the selected proposal can still increase total cached circuit states. A useful follow-up would measure end-to-end latency with controlled warm-up, identical hardware allocation, serialized model load and repeated runs, and would report the prediction, guard, GP, and exact-engine times separately. That timing experiment is not substituted for the present evaluation-count ablation, because the bake records do not contain a synchronized per-stage timer with a controlled comparator.

8 Discussion

For this suite, screening yields a striking separation between decision agreement and computational efficiency. The selected decisions are usually identical and the measured metal differences are small, yet the screened search evaluates the circuit more often. This is consistent with a mechanism in which the screen adds exact evaluations at selected candidates without removing enough ordinary polling evaluations. Close final decisions do not compensate for a 17.25% increase in the measured call count when the objective of screening is to reduce simulator work.

The result also illustrates why average prediction error is not a sufficient screen diagnostic. Optimization depends on candidate ranking near the incumbent, uncertainty calibration in the feasible region and how many candidates the screen makes the exact evaluator inspect. The observed maximum recovery error of 38.16 percentage points cautions against treating a nominal confidence-width threshold as a universal safety guarantee. Independent calibration on the exact region of use and a prospective test on new cases would be needed before relying on the screen outside this authored suite.

The comparator is deliberately narrow: the same fixed-weight multistart generalized pattern search with screening switched off. The experiment does not compare global optimization methods, alternative surrogate-management strategies or wall-clock performance on a calibrated plant model. Evaluation counts exclude the later weight path, uncached audits and surrogate fitting time. The case suite uses authored inputs and simulator-generated values, so it cannot establish transfer to unseen plants or ores.

The negative efficiency result is useful as an implementation finding because it falsifies a plausible but unverified assumption: that inserting a low-cost surrogate decision into an expensive direct search will necessarily reduce exact calls. In this implementation, screening evaluates one selected proposal exactly while the underlying poll remains available. For an additional proposal to be beneficial in the call-count sense, it must displace enough ordinary polling states or shorten the subsequent trajectory. High rejection counts alone cannot demonstrate that displacement, and a favorable endpoint alone cannot demonstrate it either. The observed counters show that the current coupling does not meet that requirement over the evaluated suite.

The close endpoint values should be interpreted with care. Most feasible decisions agree to the saved tolerance, and the remaining differences have small recovered-metal gaps in the simulator's units. This is evidence of local outcome similarity for the specified objective and constraints. It is not a statistical equivalence result: the experiment declared no decision tolerance in advance, did not calculate power for a noninferiority margin, and draws its cases from one authored suite. A future acceptance study should set a domain-informed tolerance for recovery, grade, energy and constraint margins before executing the comparison, then test it on a held-out case set. That protocol would answer a more decision-relevant question than post hoc closeness.

The largest proposal error exposes a mismatch between the meaning of the filter and the use made of it. A GP interval width is a model-dependent uncertainty quantity. It can help detect regions with weak support under that GP's training design, but width alone does not validate the ranking network, constrain error under distribution shift or certify process feasibility. The reconstruction guard also has a limited interpretation: it tests how well a learned representation reconstructs features, not whether the response function is accurate. Combining two proxies can improve a practical rule, but the combination has no automatic coverage or safety guarantee. Such guarantees require explicit assumptions, calibrated validation and a proof or a clearly scoped empirical contract.

The radius ablation is consistent with this mechanism: the one-step policy spends fewer calls than the wider stencil, yet still exceeds the no-screen baseline. This suggests that reducing candidate breadth can mitigate overhead without removing the structural cost of the current insertion point. The result motivates a different next design question: can the screen replace a poll evaluation while retaining the exact poll's role in the

search logic, or can a trust-region acceptance test use predicted decrease only where an exact model-agreement ratio supports it? Those are new methods to implement and test, not conclusions already established by this experiment.

Accordingly, the responsible engineering disposition is to retain the method as an inspectable comparison path and avoid presenting it as an accelerator. Any production use would need explicit opt-in, a visible exact-evaluation budget, a fallback poll, a record of every rejected and selected point, and a monitor that surfaces prediction error and constraint state. Since the current evidence is simulation-only and the observed filter missed a large error, it does not support autonomous plant control or operating recommendations. This is a scope boundary derived from the tested contract, not a claim that learned screening is inherently unsuitable for process optimization.

9 Limitations

All optimization outcomes are generated by an authored, steady-state process simulator. They are not measurements from operating plants. Model-form error, changing ore, plant instrumentation error, control-system latency and economic uncertainty are outside this experiment. The finite set of twelve cases does not represent the full mineral-processing design space.

The paired searches share a finite starting set, mesh and evaluation budget. Agreement is conditional on those choices and does not establish global optimality. The progressive-barrier method has established convergence analyses under their stated assumptions; this implementation's surrogate-selected search modification is not accompanied by a new convergence proof. The call-count measure also omits time spent fitting, loading and evaluating the surrogate, as well as parallel execution effects.

Internal validity is limited by the exact meaning of the counters. The saved count is the number of distinct states in each optimizer cache, not a fully instrumented count of all calls across the application. Base audits, uncached endpoint checks and weight-path calculations are outside the primary comparison. This choice is reproducible and symmetric across paired runs, but another cost contract could count every engine entry or charge for expensive preprocessing. Model loading and candidate inference are also omitted. Therefore, the 17.25 percent increase is specifically an increase in cached trial states under this accounting definition.

The deterministic implementation helps reproduce a given pair, but the study has no repeated random seeds for optimizer starts, model training or case generation. Bootstrap resampling across twelve authored cases summarizes sensitivity to those clusters; it does not replace repeated experiments or turn the case suite into a random sample of mines. The 95 percent interval should be read as a case-cluster descriptive uncertainty interval conditional on the saved suite and its design. It should not be used as a frequentist confidence statement about future operations.

Construct validity is also bounded. Recovered metal and specific energy are normalized in the search objective, and the constraints encode selected grade, power and water limits. The objective omits prices, reagent cost, tailings risk, equipment degradation, dynamic transitions and plant-level control penalties. A different operational objective could produce another candidate ranking and another interaction between screening and polling. Feasibility itself refers to the simulator's declared contract; it is not a safety assessment or engineering certification.

External validity is the largest limitation. The twelve cases are authored to span selected ore and circuit configurations and derive from the repository's process model. They are not twelve independently sampled facilities, and no operating historian was used. Surrogate training and evaluation are tied to the features, process equations and ranges represented in the release. Novel mineralogy, changing equipment, unmodeled reagent interactions and measurement noise can shift both response and uncertainty. The large observed error demonstrates that even the in-domain proposal set is not uniformly protected by the current screen; transfer claims would therefore be particularly unwarranted.

Finally, the result is version-specific. The source code, model export, case contract and optimizer settings define the experiment. A changed screen threshold, trained model, candidate ordering, exact evaluator or termination rule creates a new method variant and must produce fresh paired artifacts. Reusing these outcome numbers after such a change would break provenance. These limitations define the scope of the negative result

and the minimum conditions for a meaningful follow-on evaluation.

10 Conclusions

The neural-surrogate search step, with a reconstruction guard and Gaussian-process width filter, did not reduce recorded search evaluations on OreFlow's 72 target-mode optimization records. Its count was 17.25% higher than the paired unscreened procedure, lower in no variant, and terminal outcomes matched in 65 cases. Seven feasible endpoints differed, with at most 0.002653 t/h difference in recovered primary metal. The maximum observed proposal recovery error was 38.16 percentage points despite the uncertainty-width filter. These findings support retaining the screen as a reproducible comparison method while rejecting its use as a demonstrated compute-saving accelerator on this suite. A further evaluation would require a separately specified screening design, calibration checks on candidate regions and validation on cases not used to design the rule.

The result does not reject surrogate optimization broadly. It rejects a narrower performance claim for one exported neural screen, one GP width veto, one reconstruction guard, one candidate stencil and one finite authored suite. The next study should make the intended cost benefit explicit, measure end-to-end latency and include a comparator whose polling work can be displaced by screened decisions. It should preregister decision tolerances and evaluate on newly authored and independently sourced cases. Until those experiments show otherwise, the measured call increase and the documented tail error are the appropriate evidence for describing this implementation.

11 Reproducibility

The paired search records are stored in `data/derived/cases/`, with model exports in `models/` and the optimizer implementation under `src/oreflow/optimization/`. The experiment code and evidence generator are committed alongside these case records. Reproduction begins by installing the repository's pinned environment, running the model and artifact checks, and then rebuilding the case suite with the release pipeline. The analysis-only tables and vector figures can be regenerated with `pythonmanuscripts/surrogate-screening/generate_evidence.py`; this script reads persisted case records and the nominal radius-ablation JSON, writes `evidence.tex`, summary JSON and PDF figures, and fails when the expected 72 pairs are absent.

The principal quantities are intentionally traceable to machine-readable records. Evaluation totals come from each variant's `methods.optimization.evaluations` field and its `without_screen.evaluations` comparator. Terminal agreement is computed from exact saved decision coordinates and statuses. Proposal residuals are calculated from proposal prediction and exact engine values. The cluster interval resamples the twelve case identifiers with seed 20261102 and 10,000 draws, keeping all variants within a sampled case together. Figure and table generation uses the same records, rather than copied prose values. The separate radius ablation uses its committed measurement artifact and is not recomputed from the 72-pair table.

The filter-removal experiment is generated by `manuscripts/surrogate-screening/run_guard_ablations.py`, which loads the same exported model and four committed case configurations. Its completed JSON is an analysis input only after all twenty rows pass schema, status, uniqueness and exact-outcome checks. The manuscript's build should run the evidence generator, then pdfLaTeX, then the CAOS manuscript-depth, coverage, title-support and voice checks. The repository version, Python package versions, model checksum, case record checksum and clean process exit should accompany any archived release so a later reader can distinguish a regenerated analysis from a reused result.

The paper's computational evidence is reproducible only within the stated simulator and versioned environment. The NVIDIA GPU is used for model training in the release pipeline, while the finite optimizer and filter-removal comparisons are CPU-bound. That allocation is recorded to make resource use clear; it does not imply that GPU acceleration improves the optimizer comparison. A reproduction on different numerical libraries or processor architecture may alter floating-point ties, so exact decision identity should be checked against the documented tolerance while preserving raw outputs. No external plant data, credentials or private operational records are required to regenerate the authored-case analysis.

Data and code availability

The case-level optimization records are committed under https://github.com/fsantibanezleal/CAOS_OreFlow/tree/main/data/derived/cases; the implementation, tests, screen export and evidence-generation script are maintained in the same public repository. Each case record stores paired search counts, proposals, exact engine outcomes and terminal decisions, while model metadata documents the serialized feature order and scaling. The records are simulator outputs from authored scenarios and are not plant data. The nominal radius-ablation measurements are stored in the optimizer design measurements directory. The filter-removal experiment writes a separate machine-readable artifact whose case and policy identifiers can be checked against the script before regenerating manuscript tables. A release archive should include the exact source commit, model checksum, environment lock, generated PDFs, evidence summary and successful pipeline validation result; these identify the data and executable used to create every reported result. The public code and records are released under the repository's declared licensing terms, and their provenance remains visible at case and model level. No operating-plant observations or private credentials are required to reproduce the analyses.

References

[1] M. A. Abramson and C. Audet, "Convergence of mesh adaptive direct search to second-order stationary points," *SIAM Journal on Optimization*, vol. 17, no. 2, pp. 606–619, 2006. doi: 10.1137/050638382.

[2] C. Audet and J. E. Dennis, Jr., "A pattern search filter method for nonlinear programming without derivatives," *SIAM Journal on Optimization*, vol. 14, no. 4, pp. 980–1010, 2003. doi: 10.1137/S105262340138983X.

[3] C. Audet and J. E. Dennis, Jr., "Mesh adaptive direct search algorithms for constrained optimization," *SIAM Journal on Optimization*, vol. 17, no. 1, pp. 188–217, 2006. doi: 10.1137/040603371.

[4] A. N. Angelopoulos and S. Bates, "Conformal prediction: A gentle introduction," *Foundations and Trends in Machine Learning*, vol. 16, no. 4, pp. 494–591, 2023. doi: 10.1561/2200000101.

[5] A. J. Booker, J. E. Dennis, Jr., P. D. Frank, D. B. Serafini, V. Torczon, and M. W. Trosset, "A rigorous framework for optimization of expensive functions by surrogates," *Structural Optimization*, vol. 17, pp. 1–13, 1999. doi: 10.1007/BF01197708.

[6] G. C. Cawley and N. L. C. Talbot, "On over-fitting in model selection and subsequent selection bias in performance evaluation," *Journal of Machine Learning Research*, vol. 11, pp. 2079–2107, 2010. URL: JMLR 11 (2010).

[7] D. Eriksson, M. Pearce, J. R. Gardner, R. D. Turner, and M. Poloczek, "Scalable global optimization via local Bayesian optimization," in *Advances in Neural Information Processing Systems*, vol. 32, 2019. URL: NeurIPS 2019.

[8] R. Fletcher and S. Leyffer, "Nonlinear programming without a penalty function," *Mathematical Programming*, vol. 91, pp. 239–269, 2002. doi: 10.1007/s101070100244.

[9] A. I. J. Forrester and A. J. Keane, "Recent advances in surrogate-based optimization," *Progress in Aerospace Sciences*, vol. 45, no. 1–3, pp. 50–79, 2009. doi: 10.1016/j.paerosci.2008.11.001.

[10] T. Gneiting and A. E. Raftery, "Strictly proper scoring rules, prediction, and estimation," *Journal of the American Statistical Association*, vol. 102, no. 477, pp. 359–378, 2007. doi: 10.1198/016214506000001437.

[11] D. R. Jones, M. Schonlau, and W. J. Welch, "Efficient global optimization of expensive black-box functions," *Journal of Global Optimization*, vol. 13, pp. 455–492, 1998. doi: 10.1023/A:1008306431147.

[12] A. Kendall and Y. Gal, "What uncertainties do we need in Bayesian deep learning for computer vision?" in *Advances in Neural Information Processing Systems*, vol. 30, 2017. URL: NeurIPS 2017.

[13] E. J. Y. Koh, E. Amini, G. J. McLachlan, and N. Beaton, "Utilising a deep neural network as a surrogate model to approximate phenomenological models of a comminution circuit for faster simulations," *Minerals Engineering*, vol. 170, article 107026, 2021. doi: 10.1016/j.mineng.2021.107026.

[14] B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and scalable predictive uncertainty estimation using deep ensembles," in *Advances in Neural Information Processing Systems*, vol. 30, 2017. URL: NeurIPS 2017.

[15] A. L. Marsden, M. Wang, J. E. Dennis, Jr., and P. Moin, "Optimal aeroacoustic shape design using the surrogate management framework," *Optimization and Engineering*, vol. 5, no. 2, pp. 235–262, 2004. doi: 10.1023/B:OPTE.0000033376.89159.65.

[16] H. Papadopoulos, "Guaranteed coverage prediction intervals with Gaussian process regression," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 12, pp. 9072–9083, 2024. doi: 10.1109/TPAMI.2024.3418214.

[17] C. Audet and J. E. Dennis, Jr., "A progressive barrier for derivative-free nonlinear programming," *SIAM Journal on Optimization*, vol. 20, no. 1, pp. 445–472, 2009. doi: 10.1137/070692662.

[18] N. V. Queipo, R. T. Haftka, W. Shyy, T. Goel, R. Vaidyanathan, and P. K. Tucker, "Surrogate-based analysis and optimization," *Progress in Aerospace Sciences*, vol. 41, no. 1, pp. 1–28, 2005. doi: 10.1016/j.paerosci.2005.02.001.

[19] C. E. Rasmussen and C. K. I. Williams, *Gaussian Processes for Machine Learning*. MIT Press, 2006. URL: gaussianprocess.org/gpml.

[20] D. R. Roberts et al., "Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure," *Ecography*, vol. 40, no. 8, pp. 913–929, 2017. doi: 10.1111/ecog.02881.

[21] J. Sacks, W. J. Welch, T. J. Mitchell, and H. P. Wynn, "Design and analysis of computer experiments," *Statistical Science*, vol. 4, no. 4, pp. 409–423, 1989. doi: 10.1214/ss/1177012413.

[22] B. Shahriari, K. Swersky, Z. Wang, R. P. Adams, and N. de Freitas, "Taking the human out of the loop: A review of Bayesian optimization," *Proceedings of the IEEE*, vol. 104, no. 1, pp. 148–175, 2016. doi: 10.1109/JPROC.2015.2494218.

[23] N. Srinivas, A. Krause, S. M. Kakade, and M. Seeger, "Gaussian process optimization in the bandit setting: No regret and experimental design," in *Proceedings of the 27th International Conference on Machine Learning*, pp. 1015–1022, 2010. URL: ICML 2010.

[24] Y. Sui, A. Gotovos, J. W. Burdick, and A. Krause, "Safe exploration for optimization with Gaussian processes," in *Proceedings of the 32nd International Conference on Machine Learning*, PMLR 37, pp. 997–1005, 2015. URL: PMLR 37.

[25] V. Torczon, "On the convergence of pattern search algorithms," *SIAM Journal on Optimization*, vol. 7, no. 1, pp. 1–25, 1997. doi: 10.1137/S1052623493250780.